# Architectural Engineering and Implementation Framework for 6G Network Digital Twins with Integrated Deep Reinforcement Learning

The evolution of telecommunications toward the sixth-generation (6G) paradigm represents a departure from incremental speed improvements toward a fundamental restructuring of network intelligence. As wireless environments become increasingly complex, characterized by Terahertz (THz) frequencies, massive multi-element antenna arrays, and high-velocity mobility patterns, traditional static optimization techniques become computationally irreducible.[1] This complexity necessitates the development of Network Digital Twins (NDTs), which are high-fidelity, dynamic virtual replicas of physical network infrastructures.[4] By integrating Deep Reinforcement Learning (DRL) into these digital twins, operators can facilitate a closed-loop system where the virtual environment serves as a risk-free testbed for training autonomous agents capable of sub-millisecond resource orchestration.[7] The following report provides an exhaustive analysis of a seven-layer 6G Digital Twin architecture, the mechanisms that enable such high-fidelity synchronization, and a step-by-step engineering blueprint for implementation.

## The Multi-Layer Functional Architecture of 6G Digital Twins

The conceptualization of a 6G Digital Twin requires a stratified architecture that manages the flow of information from raw physical sensors to intelligent actuation commands. As depicted in the primary architectural framework, this system is divided into seven distinct layers, ranging from the physical network to evaluation and visualization [Image].

### Layer 0: The Physical Network and Real-Time Telemetry

At the foundation lies Layer 0, the physical network environment, which may be realized through actual 6G gNodeB base stations and User Equipment (UE) nodes or through high-fidelity discrete-event simulators such as ns-3.[2] The core responsibility of this layer is the generation of live telemetry. This telemetry encompasses spatial-temporal data points including Signal-to-Interference-plus-Noise Ratio (SINR), instantaneous throughput, handover event logs, and precise (x, y) positional coordinates.[10] In a 6G context, this also includes Integrated Communication and Sensing (ICS) data, where the network infrastructure itself acts as a sensor to perceive structural changes in the environment.[2]

The possibility of this layer relies on the deployment of software-defined radio (SDR) and

5G/6G core networks that operate in real-time, such as the OpenAirInterface (OAI) platform.[11] These systems allow for the extraction of telemetry at any point in the protocol stack, providing the "raw bits" necessary for digital twinning.[9]

## Layer 1: Data Ingestion and Feature Engineering

Raw telemetry is insufficient for direct consumption by machine learning models. Layer 1 provides the essential bridge through data ingestion and feature engineering [Image]. Data is streamed from the physical network using socket streams (often via ZeroMQ) or ingested from historical CSV logs.[12] The engineering process involves several critical sub-processes:

- **Normalization**: Scaling heterogeneous data points (e.g., SINR in dB and throughput in Mbps) into a uniform range to prevent gradient explosion in neural network training.[13]
- **Sliding Window Buffer**: Maintaining a temporal history of network states to allow the model to capture trends rather than just instantaneous snapshots.[14]
- **State Vector Construction**: Assembling the engineered features into a formal state vector. According to the architectural specification, the state vector $[s_t]$ for a 6G node includes $$ [Image].

This layer's mechanism is powered by protocol buffers (Protobuf) which ensure that data serialized in the C++ environment of the network simulator can be deserialized efficiently in a Python-based machine learning environment.[7]

## Layer 2A: The Static Digital Twin and Predictive Modeling

Layer 2A represents the "physics-aware" component of the twin. It is categorized as "Static" because it relies on established mathematical models and pre-trained supervised learning components that do not change in real-time based on RL rewards.[5]

| Component | Mathematical/Algorithmic Basis | Role in the 6G Twin |
|---|---|---|
| Physics Model | Friis Transmission Equation, ITU-R P.676 | Calculates path loss and atmospheric attenuation for THz and mmWave signals [Image]. |
| Channel Predictor | Long Short-Term Memory (LSTM) or GRU | Predicts future channel states based on historical temporal patterns.[15] |

| | | |
|---|---|---|
| Coverage Map | SINR Heatmap Grid | Provides a spatial visualization of signal quality across the deployment area [Image]. |
| Anomaly Detect | Isolation Forest | Identifies deviant network behavior (e.g., base station failures or sudden traffic surges).[8] |

The inclusion of the ITU-R P.676 model is particularly vital for 6G, as it specifically addresses attenuation by atmospheric gases, which becomes a dominant factor in the Terahertz spectrum.[2] By combining these deterministic physics models with data-driven predictors like LSTMs, the Static Digital Twin can offer a high-fidelity estimation of network behavior without the computational cost of a full simulation.[5]

## Layer 2B: The Dynamic Digital Twin and Reinforcement Learning

Layer 2B is the intellectual core of the architecture, where the network's dynamic adaptation strategies are developed. This layer utilizes an OpenAI Gym wrapper to frame the network optimization problem as a Markov Decision Process (MDP).[7]

The RL Agent specified in this framework is the Proximal Policy Optimization (PPO) algorithm, utilizing an Actor-Critic architecture and Generalized Advantage Estimation (GAE) [Image]. This choice is driven by PPO's ability to handle the complex, high-dimensional action spaces of 6G networks while maintaining training stability.[7]

The interaction between the agent and the environment is defined by three primary spaces:

1. **State Space**: The agent observes SINR, Throughput (TP), positional data, and Handover (HO) status [Image].
2. **Action Space**: The agent executes control over Beamforming (BF) gain (0-44 dBi), Transmission (TX) power control (23-46 dBm), handover triggers, and network slicing commands [Image].
3. **Reward Function**: The agent is motivated by a multi-objective reward function: $R = \alpha \cdot SINR + \beta \cdot TP - \gamma \cdot latency - \delta \cdot HO$ [Image].

A more granular reward formula, as specified in the implementation details, applies specific weights to these normalized values: $0.4 \cdot norm\_SINR + 0.3 \cdot norm\_TP - 0.2 \cdot norm\_latency - 0.1 \cdot handover\_penalty$ [Image]. This weighting reflects a 6G priority on spectral efficiency and throughput, while still penalizing excessive handovers which degrade the Quality of Experience (QoE).[2]

## Layer 3: Model Registry and Experience Replay

Layer 3 serves as the persistence and management layer. It stores the weights of the pre-trained LSTM channel models and the evolving policy weights of the PPO agent [Image]. A critical component here is the **Experience Replay Buffer**, which stores transitions $(s, a, r, s')$ to allow the agent to learn from historical successes and failures, breaking the correlation between consecutive samples in the data stream.[15]

Furthermore, this layer manages checkpointing and logging via TensorBoard. This is essential for 6G research, as it allows for the "replay" of historical network failures within the digital twin to refine the agent's resilience without impacting live users.[1]

## Layer 4: The Joint Training Pipeline

The possibility of deploying AI in a live 6G network is predicated on a phased training approach to ensure safety and reliability. The architecture defines a four-phase pipeline [5]:

- **Phase 1 (Offline Supervised)**: Models are initialized using historical datasets to learn the basic physics of the environment.[5]
- **Phase 2 (RL in Static Twin)**: The PPO agent begins learning in the virtualized environment of the Static Twin (Layer 2A), where it can explore risky actions without consequence.[5]
- **Phase 3 (RL in Live Twin)**: The agent moves to a live-synchronized twin, where it interacts with real-time telemetry but its actions are still sandboxed.[5]
- **Phase 4 (Deploy)**: Once the agent's performance meets the fidelity and safety thresholds, it is deployed to control the physical network [Image].

This phased approach mitigates the "sim-to-real gap," ensuring that the policy learned in the digital environment is robust enough for the intricacies of the physical world.[5]

## Layer 5: Control and Actuation

Layer 5 is the actuation interface where the decisions made by the RL agent are translated into physical network commands. These include beamforming steering, which is critical for 6G's massive MIMO systems to maintain high-gain links to mobile users, and TX power control to minimize inter-cell interference.[2] Additionally, this layer manages handover triggers and network slicing commands, allowing the network to dynamically partition resources for different service classes (e.g., eMBB vs. URLLC).[2]

## Layer 6: Evaluation and Visualization

The final layer provides the feedback loop necessary for continuous improvement. It measures "Twin Fidelity" using Mean Squared Error (MSE) and Mean Absolute Error (MAE) to ensure the digital replica accurately mirrors the physical network.[13] If the MSE exceeds a certain threshold, it indicates that the twin is no longer synchronized, triggering a model retraining or data

re-calibration.[13]

Visualization tools such as Matplotlib live plots, SINR Cumulative Distribution Function (CDF) curves, and RL convergence graphs provide researchers with the insights needed to tune the hyperparameters of the system.[2]

# Mechanism of Synchronization: Age of Digital Twin (AoDT)

The "how" of this architecture's possibility is fundamentally tied to the synchronization frequency between the physical and digital layers. In 6G, this is governed by the metric of Age of Information (AoI) or, more specifically, the Age of Digital Twin (AoDT).[18]

$$AoDT = t - t_{update}$$

Where $t$ is the current time and $t_{update}$ is the timestamp of the last synchronized state.[18] Maintaining a low AoDT is critical for the RL agent; if the state information is stale, the action taken by the agent (e.g., a beam steering command) will be based on an outdated user position, leading to dropped connections.[18] To solve the synchronization problem under budget constraints, 6G architectures often employ Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithms to optimize the positioning of aerial nodes or the frequency of telemetry updates, ensuring that high-fidelity reflections are maintained even in dynamic environments.[18]

# Implementation Options and Toolchains

There are two primary pathways for implementing the architecture described above, depending on whether the researcher prioritizes protocol-stack depth or physical-layer accuracy.

## Option 1: The ns-3 and ns3-gym Ecosystem (Standard Academic Approach)

This option utilizes the ns-3 discrete-event network simulator integrated with OpenAI Gym.[7] It is the de-facto standard for studying protocol operations and resource management.[2]

- **Simulator**: ns-3 (with 5G-LENA or mmWave modules).[2]
- **RL Integration**: ns3-gym framework.[7]
- **Communication**: ZeroMQ and Protobuf for C++/Python interoperability.[12]
- **Analytics**: TensorBoard and Matplotlib.[12]

This path is ideal for the architecture's Layer 2B and Layer 3, where the focus is on the RL agent's training cycles and the interaction between network layers (MAC/PHY).[2]

### Option 2: NVIDIA Aerial and Sionna (High-Fidelity Physical Twin)

For research focusing on the physics of the environment (Layer 2A), NVIDIA's toolchain provides a "Three-Computer Solution" consisting of design/training, simulation, and field deployment.[9]

- **Ray Tracing**: NVIDIA Sionna RT for 3D, deterministic channel modeling.[22]
- **Digital Twin Platform**: NVIDIA Aerial Omniverse Digital Twin (AODT).[9]
- **Hardware Acceleration**: GPU-accelerated LDPC decoding and neural receivers using CUDA and Tensor Cores.[9]

This option is superior for creating the "Static Twin" components like SINR heatmaps and physics-compliant 3D scenes, as it can calculate city-scale RF environments in milliseconds.[9]

## Step-by-Step Implementation of the 6G Digital Twin RL Architecture

The following steps outline the implementation of the seven-layer architecture using the ns-3 and ns3-gym framework, as it is most closely aligned with the provided architectural diagram.

### Phase 1: Environment Preparation and Dependency Management

The complexity of the system requires a rigid set of dependencies to ensure the ZMQ-based communication layer functions correctly across C++ and Python environments.

1. **System Configuration**: Deploy a Linux-based environment (Ubuntu 22.04 recommended). Ensure python3, pip3, and g++ are installed.[12]
2. **Compulsory Libraries**: Install the messaging and serialization libraries:
   - sudo apt-get install libzmq5 libzmq5-dev
   - sudo apt-get install libprotobuf-dev protobuf-compiler.[12]
3. **ns-3 Installation**: Clone the ns-3 repository (version 3.36 or higher is required for modern 5G/6G modules). Move the ns3-gym source to the contrib/opengym directory.[12]
4. **Framework Compilation**: Configure the project with examples enabled: ./ns3 configure --enable-examples. Build the project using ./ns3 build. During this process, the OpenGym protocol buffer messages for both C++ and Python are generated.[12]
5. **Python Module Installation**: Navigate to contrib/opengym/model/ns3gym and install the Python module: pip3 install..[12]

### Phase 2: Modeling the Physical Twin (Layer 0 & 1)

In this phase, the actual network scenario is defined within an ns-3 script.

1. **Node Initialization**: Use NodeContainer to create gNBs and UEs. For 6G scenarios, include satellite or UAV nodes using the NTN (Non-Terrestrial Network) modules.[2]
2. **Spectrum Configuration**: Utilize the MmWaveHelper or 5G-LENA spectrum modules.

Set the frequency standards (e.g., 28 GHz for mmWave or higher for THz research).[2]

3. **Mobility Assignment**: Attach mobility models to nodes. Use ConstantVelocityMobilityModel for UEs to simulate realistic movement patterns.[2]
4. **Telemetry Gateway**: Instantiate the OpenGymGateway object in the ns-3 script. This object acts as the interface for Layer 1, collecting the SINR, throughput, and position data from the simulation and packaging it for the agent.[7]

## Phase 3: Building the Static Twin and Predictive Models (Layer 2A)

This phase focuses on the "First-Principles" and anomaly detection components.

1. **Physics Models**: Integrate the Friis and ITU-R P.676 models within the ns-3 channel modeling classes. If using Sionna, this is handled via deterministic ray tracing.[22]
2. **Supervised Training**: Train an LSTM-based channel predictor using historical telemetry. This model is saved in the **Model Registry** (Layer 3) and used at runtime to provide the RL agent with "future" state estimations.[15]
3. **Anomaly Detection**: Implement an Isolation Forest algorithm in Python. This model monitors the telemetry stream; if it detects a deviation (e.g., a "Sandwich" attack or unexpected congestion), it sends an alert to the RL agent to switch to a more conservative policy.[8]

## Phase 4: Dynamic RL Agent Development (Layer 2B)

The RL agent is developed using standard Python libraries such as Stable Baselines3 or Ray RLLib.

1. **Environment Wrapper**: Create a Python class that inherits from gym.Env. This class defines how to interpret the ZMQ messages from ns-3 as observation and reward.[7]
2. **Defining the PPO Agent**:
   ```
   Python
   from stable_baselines3 import PPO
   model = PPO("MlpPolicy", env, verbose=1, gae_lambda=0.95, clip_range=0.2)
   ```
   The Actor-Critic architecture is implicitly defined within the MLP policy.
3. **Reward Integration**: Code the specific weighted reward function:
   $$R = 0.4 \cdot SINR_{norm} + 0.3 \cdot TP_{norm} - 0.2 \cdot Latency_{norm} - 0.1 \cdot HO\_Penalty$$
   [Image].
4. **Action Mapping**: Map the agent's discrete or continuous output to physical commands:
   - Action 0-10: Adjust TX Power in 2dB increments.
   - Action 11-20: Select Beamforming codebook index.[15]

## Phase 5: Executing the Training Pipeline (Layer 4)

The training is executed in two terminal processes to facilitate the simulator-agent interaction.

1. **Start the Simulator**: In the first terminal, run the ns-3 script. The OpenGymGateway will open a ZMQ port (default 5555) and wait for a connection.[12]
2. **Start the Agent**: In the second terminal, run the Python script. The agent will connect to the simulator, receive the initial state vector $$, and begin the training loop.[12]
3. **Phase Transition**:
    - **Phase 2**: Train for $10^6$ steps in the static environment.
    - **Phase 3**: Connect the agent to a "live" telemetry stream from an SDR or a high-fidelity emulator to fine-tune the policy weights.[5]

### Phase 6: Actuation, Evaluation, and Feedback (Layer 5 & 6)

1. **Actuation**: Ensure that the actions sent by the RL agent are correctly interpreted by the ns-3 NetDevice. For example, a "handover" action should trigger the RrcConnectionReconfiguration procedure in the LTE/NR stack.[2]
2. **Evaluation**: During execution, Layer 6 calculates the **Twin Fidelity**. If the virtual SINR predicted by the Static Twin (Layer 2A) differs from the "Real" SINR (Layer 0) by more than a defined MSE threshold, the simulation state is updated to reflect the new physical reality.[13]
3. **Visualization**: Launch the KPI dashboard. Monitor the SINR CDF and the RL convergence curves. Successful implementation should show the reward increasing and stabilizing over time, while the latency remains below the 1ms 6G target.[2]

## Nuanced Insights: The Role of Semantic and Goal-Oriented Twinning

As 6G research progresses, a significant trend is moving away from "Full-Scale Fidelity" toward "Semantic and Goal-Oriented" digital twins (GOST).[19] The standard architecture shown in the figure pursues physical mirroring, but in massive SAGSIN (Space-Air-Ground-Sea Integrated Networks), this becomes computationally prohibitive.[5]

Goal-oriented principles prioritize "utility" over "fidelity." In this paradigm, Layer 2A might only model the environment to the level of detail required for a specific task (e.g., beamforming) rather than a full environmental reconstruction.[19] This lightweight modeling approach reduces the telemetry overhead and allows for much faster synchronization (lower AoDT), which is critical for the real-time closed-loop control required by the RL agent in Layer 2B.[18]

## Conclusion: Future Outlook and Actionable Recommendations

The integration of Digital Twins and Reinforcement Learning in 6G is not merely a simulation

enhancement but a fundamental requirement for operating at the edge of physical possibility. The 7-layer architecture provides a robust roadmap for this integration. For professional implementation, it is recommended to:

- **Prioritize Synchronization**: Use AoDT metrics to dynamically adjust telemetry reporting rates, ensuring the RL agent always operates on fresh data.[18]
- **Adopt Phased Training**: Never deploy an RL agent directly to a live 6G network without extensive "Risk-Free" training in a Static Twin (Phase 2) and verification in a Live Twin (Phase 3).[5]
- **Utilize Hybrid Modeling**: Combine the deterministic accuracy of physics-based models (Friis/ITU) with the predictive power of LSTMs to create a "best-of-both-worlds" Static Twin.[15]

By following the step-by-step framework and leveraging the high-fidelity tools of the ns-3 and NVIDIA ecosystems, researchers can bridge the gap between theoretical 6G aspirations and operational network intelligence.

### Works cited


1. RAN Digital Twins: Optimizing Network Performance | PDF ... - Scribd, accessed April 17, 2026, https://www.scribd.com/document/814779159/digital-twin
2. steps to Begin Implementing a 6G Networks projects using ns3, accessed April 17, 2026, https://ns3-code.com/how-to-begin-implementing-a-6g-networks-projects-using-ns3/
3. (PDF) A Comprehensive Survey on Revolutionizing Connectivity Through Artificial Intelligence-Enabled Digital Twin Network in 6G - ResearchGate, accessed April 17, 2026, https://www.researchgate.net/publication/379558427_A_Comprehensive_Survey_on_Revolutionizing_Connectivity_Through_Artificial_Intelligence-Enabled_Digital_Twin_Network_in_6G
4. An Architectural Framework for 6G Network Digital Twins System, accessed April 17, 2026, https://pure.uva.nl/ws/files/244502424/3636534.3696730.pdf
5. Network Digital Twin for 6G and Beyond: An End-to ... - IEEE Xplore, accessed April 17, 2026, https://ieeexplore.ieee.org/iel8/8782661/10829557/11130513.pdf
6. Digital Network Twins for Next-Generation Wireless: Creation, Optimization, and Challenges, accessed April 17, 2026, https://arxiv.org/html/2410.18002v2
7. Architecture of ns3-gym framework. | Download Scientific Diagram - ResearchGate, accessed April 17, 2026, https://www.researchgate.net/figure/Architecture-of-ns3-gym-framework_fig2_337512642
8. A Comprehensive Survey on Revolutionizing Connectivity Through Artificial Intelligence-Enabled Digital Twin Network in 6G - IEEE Xplore, accessed April 17, 2026, https://ieeexplore.ieee.org/iel7/6287639/10380310/10488409.pdf
9. Improve AI-Native 6G Design with the NVIDIA Aerial Omniverse ..., accessed April

17, 2026, https://developer.nvidia.com/blog/improve-ai-native-6g-design-with-the-nvidia-aerial-omniverse-digital-twin/
10. Steps to implement Network Digital Twins in ns3 - NS3 Simulation, accessed April 17, 2026, https://ns3simulation.com/how-to-implement-network-digital-twins-in-ns3/
11. Powering AI-Native 6G Research with the NVIDIA Sionna Research Kit, accessed April 17, 2026, https://developer.nvidia.com/blog/powering-ai-native-6g-research-with-the-nvidia-sionna-research-kit/
12. ns3-gym: OpenAI Gym integration - ns-3 App Store, accessed April 17, 2026, https://apps.nsnam.org/app/ns3-gym/
13. Digital Twin Applications in Renewable Energy: A Survey on Structural Types, Modeling Approaches, and Functional Layers - ResearchGate, accessed April 17, 2026, https://www.researchgate.net/publication/401577032_Digital_Twin_Applications_in_Renewable_Energy_A_Survey_on_Structural_Types_Modeling_Approaches_and_Functional_Layers
14. (PDF) Fully learnable deep wavelet transform for unsupervised monitoring of high-frequency time series - ResearchGate, accessed April 17, 2026, https://www.researchgate.net/publication/358717327_Fully_learnable_deep_wavelet_transform_for_unsupervised_monitoring_of_high-frequency_time_series
15. An Improved Reinforcement Learning-Based 6G UAV Communication for Smart Cities, accessed April 17, 2026, https://www.techscience.com/cmc/v86n1/64490/html
16. AI and Digital Twins in 6G Networks Training Course - NobleProg Nepal, accessed April 17, 2026, https://www.nobleprog-np.com/cc/aidt6g
17. Digital Network Twins for Next-generation Wireless: Creation, Optimization, and Challenges, accessed April 17, 2026, https://arxiv.org/html/2410.18002v1
18. Budget-Constrained Digital Twin Synchronization and Its Application on Fidelity-Aware Queries in Edge Computing | Request PDF - ResearchGate, accessed April 17, 2026, https://www.researchgate.net/publication/383835148_Budget-Constrained_Digital_Twin_Synchronization_and_Its_Application_on_Fidelity-Aware_Queries_in_Edge_Computing
19. Twinning for Space-Air-Ground-Sea Integrated Networks: Beyond Conventional Digital Twin Towards Goal-Oriented Semantic Twin - arXiv, accessed April 17, 2026, https://arxiv.org/pdf/2512.16058
20. AoI-Aware User Service Satisfaction Enhancement in Digital Twin-Empowered Edge Computing - CS @ CityU, accessed April 17, 2026, https://www.cs.cityu.edu.hk/~weliang/papers/LGLWCXX24.pdf
21. [1810.03943] ns3-gym: Extending OpenAI Gym for Networking Research - arXiv, accessed April 17, 2026, https://arxiv.org/abs/1810.03943
22. Bringing Real-Time Ray-Tracing to ns-3: Integrating Sionna RT into ..., accessed April 17, 2026, https://5g-lena.cttc.es/blog/33/

23. robpegurri/ns3-rt: ns-3 module with NVIDIA Sionna RT as channel model - GitHub, accessed April 17, 2026, https://github.com/robpegurri/ns3-rt
24. BMS Institute of Technology | Bengaluru, India | BMSIT - ResearchGate, accessed April 17, 2026, https://www.researchgate.net/institution/BMS_Institute_of_Technology